As I am migrating from an older system, the membership date of a member may extend far in the past. Since contribution data may not be available, new system will start the membership with an opening balance for the member's cash and fund accounts, reflected accordingly into the master cash and fund accounts. Since these past cycle will be flagged as missed cycles leading to delinquency, I need a workflow to resolve these cycles.

Here is the full migration cycle resolution workflow, built across six focused sections.

1 · Member onboarding
Establishing opening balances and the migration snapshot — the foundation before any cycle resolution begins.

Collect migration data per member

From the old system or manual records, gather: join date, cash balance at migration date, fund balance at migration date, and any known history flags. If no historical data exists, balances default to zero and the migration date is treated as the effective start.

Post opening balance — Member Cash Account

Credit the member's cash account with the migrated cash balance. Simultaneously debit the Master Cash Account to reflect the system-wide cash position.

Journal — opening cash balance

DR — Master Cash AccountCash opening balance

CR — Member Cash AccountCash opening balance

Post opening balance — Member Fund Account

Credit the member's fund account with the migrated fund balance. Debit the Master Fund Account. This establishes the member's equity position in the fund from day one of the new system.

Journal — opening fund balance

DR — Master Fund AccountFund opening balance

CR — Member Fund AccountFund opening balance

Both entries are tagged `MIGRATION_OPENING_BALANCE` with the migration effective date (not the posting date). This distinguishes them from regular contributions in all reporting.

Set the migration cutoff date

Record `migration_cutoff_date` — the date from which the new system takes over. The member's join date and the cutoff together define the historical window to generate cycle stubs for.

The cutoff date is immutable once set. Changing it invalidates all cycle resolution records. Admin approval required to amend.

Generate historical cycle stubs

System auto-generates one cycle record per month between `join_date` and `migration_cutoff_date`. Each stub is created with status UNRESOLVED — not MISSED. This is a critical distinction: UNRESOLVED means "origin unknown", whereas MISSED means "known obligation not paid" and would trigger delinquency rules immediately.

Do not mark historical cycle stubs as MISSED at creation. This would trigger the late fee engine and delinquency flags against the member before any resolution process has run.

Member profile status → MIGRATION_PENDING

While in this status: the late fee engine is suppressed, delinquency rules do not apply, and guarantor notifications cannot fire. The status lifts to ACTIVE only after all historical cycles are resolved and admin grants clearance (see Tab 4).

2 · Cycle classification
Admin-driven review of each UNRESOLVED stub — deciding what each past cycle actually represents before any financial action is taken.

Classification options

Classification

Meaning

System action

WAIVED

Cycle predates available records; admin grants a full waiver

Cycle closed with zero obligation. No debit, no fee. Audit entry only.

BACKDATED_PAID

Evidence exists that member paid in the old system

Cycle marked settled. Opening balance already reflects this. No additional debit.

BACKDATED_DUE

Cycle was genuinely missed; obligation exists

Cycle converted to payable. Must be resolved via a method in Tab 3.

OB_ABSORBED

Opening fund balance already accounts for these contributions

Cycle batch-closed. No debit. Opening balance flagged as settlement vehicle.

ESCALATED

Disputed or requires further investigation

Cycle frozen. Does not count toward delinquency while in this state.

---

Batch vs individual classification

Batch classification (recommended for long histories)

Admin sets a policy rule per member or cohort: e.g. “all cycles before year Y are WAIVED; cycles from year Y onward require individual review.” System applies the rule across all matching stubs in one operation.

Individual classification

Admin reviews each cycle stub one by one. Suitable when partial records exist in the old system and specific cycles can be matched to payment evidence from the legacy data.

---

Opening balance absorption — when to use it

If the migrated fund opening balance was computed to already represent cumulative contributions (i.e. the old system tracked a running fund total per member), historical cycles should be classified OB_ABSORBED in bulk. This avoids double-counting — the opening balance entry already credits what those cycles would have contributed.

Never use OB_ABSORBED if the opening balance was an estimate or approximation. Use WAIVED instead, and record the basis of the opening balance in the audit entry.

3 · Resolution methods
Three paths for settling genuine historical obligations classified as BACKDATED_DUE — all requiring explicit admin selection per member.

Method A — Lump-sum settlement

Member pays all outstanding backdated cycles in a single transaction. Total due = sum of all BACKDATED_DUE contribution amounts across the historical window. Admin posts the settlement; all affected cycles are marked SETTLED simultaneously.

Journal — lump-sum backdated settlement

DR — Member Cash AccountTotal backdated amount

CR — Master Fund AccountTotal backdated amount

CR — Member Fund AccountTotal backdated amount (memo)

Fastest path to clearing MIGRATION_PENDING. Recommended when the member has sufficient cash account balance to cover the full obligation.

Method B — Instalment plan (spread over future cycles)

Admin defines a schedule: the total backdated amount is divided into equal instalments collected over N future cycles, running alongside — but separate from — regular

contributions. Each instalment uses the same collection mechanism and late fee rules as a standard EMI if missed.

Journal — per instalment collected

DR — Member Cash AccountInstalment amount

CR — Master Fund AccountInstalment amount

CR — Member Fund AccountInstalment amount (memo)

Unlike an active loan, the member is NOT automatically exempt from regular contributions during an instalment plan. Both obligations run in parallel unless admin explicitly grants a concurrent exemption.

Method C — Opening balance offset

If the member's opening fund balance exceeds total backdated obligations, admin may offset the historical cycles directly against it. The member's fund account is debited, the master fund is credited — no cash movement required.

Journal — opening balance offset

DR — Member Fund AccountTotal backdated amount

CR — Master Fund AccountTotal backdated amount

Only applicable when opening fund balance ≥ total backdated obligations. Results in a reduced member fund balance. Member must be informed and consent recorded before admin posts this.

Hybrid — partial waiver + partial settlement

Admin may apply different classifications and methods to different date ranges within a single member's history. Example: cycles before year Y → WAIVED; cycles year Y to cutoff → Method B instalment. Each range is handled independently and produces separate audit records.

---

Late fee treatment for backdated cycles

Late fees are never automatically applied to BACKDATED_DUE cycles, regardless of how long they have been outstanding. The late fee engine is suppressed for all cycles tagged with origin `MIGRATION`. If admin determines fees are appropriate for a specific obligation, they must be applied manually with an explicit override reason recorded in the audit log.

4 · Delinquency clearance
Conditions and steps for lifting MIGRATION_PENDING and releasing the member into normal operating status.

Clearance conditions (all must be met)

Condition

Required?

Notes

No UNRESOLVED stubs remain

Mandatory

Every stub must have a terminal classification

All BACKDATED_DUE cycles have a resolution method assigned

Mandatory

Method A, B, or C must be selected and initiated

No ESCALATED cycles remain open

Mandatory

Cannot clear while any cycle is frozen in ESCALATED state

Lump-sum or offset fully posted (if Method A or C)

Conditional

Cash must be posted before clearance if Method A or C chosen

Instalment schedule built and first date set (if Method B)

Conditional

Schedule must exist; first instalment does not need to have been collected yet

Admin sign-off

Mandatory

Clearance is not automatic — requires explicit admin approval action

---

Clearance flow

System pre-clearance validation

Before the admin approval action is enabled, the system validates all mandatory conditions. Any unmet condition blocks the clearance button and surfaces a checklist showing which items remain outstanding.

Admin reviews and approves clearance

Admin confirms the resolution summary: cycles waived, cycles settled, cycles on instalment, opening balance offsets applied. Admin records a clearance note and submits approval. This creates the immutable clearance record.

Member status → ACTIVE

MIGRATION_PENDING is lifted. The member enters the normal operating cycle from the next collection period. If an instalment plan is active, it runs in parallel with regular contributions from this point forward.

From this cycle: contribution collection applies, the late fee engine activates, loan eligibility is evaluated normally, and all delinquency rules take effect.

Member notified

Member receives a migration clearance summary: opening balances confirmed, historical cycles resolved by type, any ongoing instalment obligations, and the date from which normal operating rules apply.

---

Partial clearance (advanced)

For members with very long histories, admin may grant partial clearance: ACTIVE status is issued for all new-cycle operations while a small subset of ESCALATED cycles continues to be investigated in the background. The escalated cycles are ring-fenced and cannot affect operational status once partial clearance is issued. This requires an explicit flag: `PARTIAL_CLEARANCE_GRANTED`.

5 · Journals & audit
All journal entries required for migration, tagged to distinguish them from live operational postings, plus mandatory audit trail fields.

Full journal entry reference

Event

DR

CR

Tag

Opening cash balance

Master Cash Account

Member Cash Account

`MIGRATION_OPENING`

Opening fund balance

Master Fund Account

Member Fund Account

`MIGRATION_OPENING`

Cycle waiver

—

—

`MIGRATION_WAIVER` — zero entry, audit log only

Backdated paid (evidence-based)

—

—

`MIGRATION_BACKDATED_PAID` — no cash movement

OB absorbed

—

—

`MIGRATION_OB_ABSORBED` — no cash movement

Lump-sum settlement

Member Cash Account

Master Fund + Member Fund (memo)

`MIGRATION_LUMPSUM`

Instalment collected

Member Cash Account

Master Fund + Member Fund (memo)

`MIGRATION_INSTALMENT`

Opening balance offset

Member Fund Account

Master Fund Account

`MIGRATION_OB_OFFSET`

Manual late fee (override only)

Member Cash Account

Late Fee Income Account

`MIGRATION_MANUAL_FEE`

---

Audit trail mandatory fields

Per cycle resolution record

`cycle_id, member_id, original_cycle_date, classification, resolution_method, admin_id, resolution_date, evidence_ref, notes`

Per journal entry

`transaction_id, entry_type (MIGRATION_* tag), effective_date (historical), posting_date (actual), admin_id, migration_batch_id, linked_cycle_ids[]`

The `effective_date` field is critical for reporting — it allows historical contribution reports to show correct period totals even though transactions were physically posted during migration. Effective date = the month the cycle represents. Posting date = when the entry was created in the new system.

---

Master account reconciliation — migration integrity check

ASSERT:
Master Fund Account balance == Σ(all Member Fund Account balances) + Σ(all

BACKDATED_DUE obligations not yet collected)
ASSERT:
Master Cash Account balance == Σ(all Member Cash Account balances)
IF either assertion fails:
flag MIGRATION_RECONCILIATION_ERROR block clearance for all affected members
require admin investigation before proceeding

6 · Algorithms

1 · Historical cycle stub generation

```
FUNCTION generate_historical_stubs(member): cycle = first_cycle_month(member.j
```

2 · Batch classification

```
FUNCTION batch_classify(member, date_range, classification, method=null): stub
```

3 · Instalment schedule builder

```
FUNCTION build_instalment_schedule(member, due_stubs): total_due = sum(s.amour
```

4 · Clearance eligibility check

```
RATION_CLEARANCE_GRANTED', member, admin) notify(member, 'migration_complete')
```